Студент Рукленок Степан Алексеевич

Группа САУ-26

**Решение**

```python
import pandas as pd
import numpy as np
import os

# 1. ЧТЕНИЕ ДАННЫХ
print("ПУНКТ 1. ЧТЕНИЕ ДАННЫХ")
try:
    if os.path.exists('Coffee Shop Sales.xlsx'):
        df = pd.read_excel('Coffee Shop Sales.xlsx', sheet_name='Transactions')
        print("Данные загружены из Excel")
    else:
        df = pd.read_csv('Coffee Shop Sales.csv')
        print("Данные загружены из CSV")
except FileNotFoundError:
    print("Ошибка: Файл не найден.")
    exit()

print(df.head())
print(df.info())
print(df.describe())
print("\nВЫВОД: Данные успешно прочитаны. В исходном файле в колонке unit_price есть тексто


# 2. ФИЛЬТРАЦИЯ
print("ПУНКТ 2. ФИЛЬТРАЦИЯ")
threshold = 3.0
category = 'Coffee'

filtered = df[(df['unit_price'] > threshold) & (df['product_category'] == category)]

print(f"Порог: {threshold}, Категория: {category}")
print(f"Количество строк: {len(filtered)}")
print(filtered.head())
print("\nВЫВОД: Фильтрация выполнена.\n")

```

```
35
36
37  # 3. ГРУППИРОВКА И АГРЕГАЦИЯ
38  print("ПУНКТ 3. ГРУППИРОВКА И АГРЕГАЦИЯ")
39  grouped = df.groupby('product_category').agg(
40      transaction_count=('transaction_id', 'count'),
41      avg_price=('unit_price', 'mean'),
42      total_qty=('transaction_qty', 'sum'),
43      max_price=('unit_price', 'max')
44  ).sort_values('avg_price', ascending=False)
45
46  print(grouped)
47  top_category = grouped.index[0]
48  print(f"\nВЫВОД: Самая высокая средняя цена у категории '{top_category}'.\n")
49
50
51  # 4. НОВЫЙ СТОЛБЕЦ
52  print("ПУНКТ 4. НОВЫЙ СТОЛБЕЦ price_level")
53  avg_price_all = df['unit_price'].mean()
54  print(f"Средняя цена по таблице: {avg_price_all:.2f}")
55
56  df['price_level'] = np.where(
57      df['unit_price'] > avg_price_all,
58      'выше среднего',
59      'ниже или равно'
60  )
61  print(df[['unit_price', 'price_level']].head())
62  print("\nВЫВОД: Столбец добавлен.\n")
63
```

```python

# 5. ИСПОЛЬЗОВАНИЕ NUMPY
print("ПУНКТ 5. ИСПОЛЬЗОВАНИЕ NUMPY")
# Так как в колонке есть текст, NumPy при конвертации выдаст строки.
# Чтобы не было ошибки, используем pd.to_numeric с параметром errors='coerce'
# (это базовая защита, чтобы программа не упала).
prices = pd.to_numeric(df['unit_price'], errors='coerce').dropna().to_numpy()

print(f"Среднее: {prices.mean():.2f}")
print(f"Стандартное отклонение: {prices.std():.2f}")
print(f"Максимум: {prices.max():.2f}")
print(f"Минимум: {prices.min():.2f}")

normalized = (prices - prices.min()) / (prices.max() - prices.min())
print("Первые 5 нормализованных значений:", normalized[:5])
print("\nВЫВОД: Статистика рассчитана, нормализация выполнена.\n")


# 6. СОХРАНЕНИЕ
print("ПУНКТ 6. СОХРАНЕНИЕ")
# Сохраняем с кодировкой и разделителем для красивого открытия в Excel
df.to_csv('processed_coffee.csv', index=False, encoding='utf-8-sig', sep=';')
print("Файл processed_coffee.csv сохранен.")


# ОТВЕТЫ НА ВОПРОСЫ
print("\nОТВЕТЫ НА ВОПРОСЫ:")
print(f"1. Сколько строк осталось после фильтрации: {len(filtered)}")
print(f"2. Какая категория имеет самую высокую среднюю цену: {top_category}")
```

**Ответ:**

```
ПУНКТ 1. ЧТЕНИЕ ДАННЫХ
Данные загружены из Excel
   transaction_id  ...              product_detail
0               1  ...                  Ethiopia Rg
1               2  ...     Spicy Eye Opener Chai Lg
2               3  ...           Dark chocolate Lg
3               4  ...  Our Old Time Diner Blend Sm
4               5  ...     Spicy Eye Opener Chai Lg

[5 rows x 11 columns]
<class 'pandas.DataFrame'>
RangeIndex: 149116 entries, 0 to 149115
Data columns (total 11 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   transaction_id    149116 non-null  int64
 1   transaction_date  149116 non-null  datetime64[us]
 2   transaction_time  149116 non-null  object
 3   transaction_qty   149116 non-null  int64
 4   store_id          149116 non-null  int64
 5   store_location    149116 non-null  str
 6   product_id        149116 non-null  int64
 7   unit_price        149116 non-null  float64
 8   product_category  149116 non-null  str
 9   product_type      149116 non-null  str
 10  product_detail    149116 non-null  str
dtypes: datetime64[us](1), float64(1), int64(4), object(1), str(4)
memory usage: 12.5+ MB
```

```
dtypes: datetime64[us](1), float64(1), int64(4), object(1), str(4)
memory usage: 12.5+ MB
None
       transaction_id           transaction_date  ...     product_id     unit_price
count   149116.000000                     149116  ...  149116.000000  149116.000000
mean     74737.371872  2023-04-15 11:50:32.173609  ...      47.918607       3.382219
min          1.000000        2023-01-01 00:00:00  ...       1.000000       0.800000
25%      37335.750000        2023-03-06 00:00:00  ...      33.000000       2.500000
50%      74727.500000        2023-04-24 00:00:00  ...      47.000000       3.000000
75%     112094.250000        2023-05-30 00:00:00  ...      60.000000       3.750000
max     149456.000000        2023-06-30 00:00:00  ...      87.000000      45.000000
std      43153.600016                        NaN  ...      17.930020       2.658723

[8 rows x 6 columns]

ВЫВОД: Данные успешно прочитаны. В исходном файле в колонке unit_price есть текстовые значения, но Pandas корректно обрабатывает их при расчетах.

ПУНКТ 2. ФИЛЬТРАЦИЯ
Порог: 3.0, Категория: Coffee
Количество строк: 22468
    transaction_id transaction_date  ...          product_type  product_detail
8                9       2023-01-01  ...       Barista Espresso       Latte Rg
11              12       2023-01-01  ...  Gourmet brewed coffee     Ethiopia Lg
20              21       2023-01-01  ...  Gourmet brewed coffee     Ethiopia Lg
24              25       2023-01-01  ...       Barista Espresso      Cappuccino
29              30       2023-01-01  ...       Barista Espresso      Cappuccino

[5 rows x 11 columns]

ВЫВОД: Фильтрация выполнена.
```

```
ПУНКТ 3. ГРУППИРОВКА И АГРЕГАЦИЯ
|   |   |   |   |   transaction_count  avg_price  total_qty  max_price
product_category
Coffee beans                     1753  21.018397       1828      45.00
Branded                           747  17.720214        776      28.00
Loose Tea                        1210   9.267438       1210      10.95
Packaged Chocolate                487   9.050595        487      13.33
Drinking Chocolate              11468   4.148827      17457       4.75
Bakery                          22796   3.551682      23214       5.63
Coffee                          58416   3.023646      89250       4.25
Tea                             45449   2.817121      69737       4.00
Flavours                         6790   0.800000      10511       0.80

ВЫВОД: Самая высокая средняя цена у категории 'Coffee beans'.

ПУНКТ 4. НОВЫЙ СТОЛБЕЦ price_level
Средняя цена по таблице: 3.38
|   unit_price     price_level
0         3.0  ниже или равно
1         3.1  ниже или равно
2         4.5   выше среднего
3         2.0  ниже или равно
4         3.1  ниже или равно

ВЫВОД: Столбец добавлен.
```

```
ПУНКТ 5. ИСПОЛЬЗОВАНИЕ NUMPY
Среднее: 3.38
Стандартное отклонение: 2.66
Максимум: 45.00
Минимум: 0.80
Первые 5 нормализованных значений: [0.04977376 0.0520362  0.08371041 0.02714932 0.0520362 ]

ВЫВОД: Статистика рассчитана, нормализация выполнена.

ПУНКТ 6. СОХРАНЕНИЕ
Файл processed_coffee.csv сохранен.

ОТВЕТЫ НА ВОПРОСЫ:
1. Сколько строк осталось после фильтрации: 22468
2. Какая категория имеет самую высокую среднюю цену: Coffee beans
```